

1. Relational databases:

	Relational	object - relational
Complexity	text only	Can represent multimedia
indexing	faster indexing since the queries ^{are simple}	Slower indexing since the query is more complex
request predicates	exact / attribute based	attribute - based + complex
advantage	only supports primitive data types	supports primitive and user defined
disadvantage	easier to implement and more performant	harder to implement because of complexity

2. User defined data types = defining complex types like images, video, audio with their own attributes and methods

Inheritance = defining general media type and deriving subtypes that automatically reuse its attribute and behavior, ex: "Video" can inherit from "media" type

Polymorphism = Same query can work on different types, ex: extract_audio() can work on video and audio

Object identifiers = each objects must have unique system generated identifiers (like primary key, but immutable and system-managed)

Table of objects = tables that stores instances of user defined data types
ex: Address type (Street, City, zip) then create table customers of Customer type

Relations between Uddt = 1:1 relation uses single foreign-key reference from one object to another, 1:n store a collections in the parent object ex: Users have many orders but order can only belongs to one user

REF: Unique foreign-key like reference from object to other, ex: an annotation within an image will always reference the original image

DEREF: a reference that points to the actual object it refers from
ex: Referencing "Sunset" annotation that refers to an actual image with a sunset in it

Collections: a list inside one field. ex: Students has many courses followed with collections we can store all the courses within one column/attribute

Dot-operator: a way to access an attribute of an object, example:

Select i.width, i.extract_features()

From images i

Where i.type = 'picture';

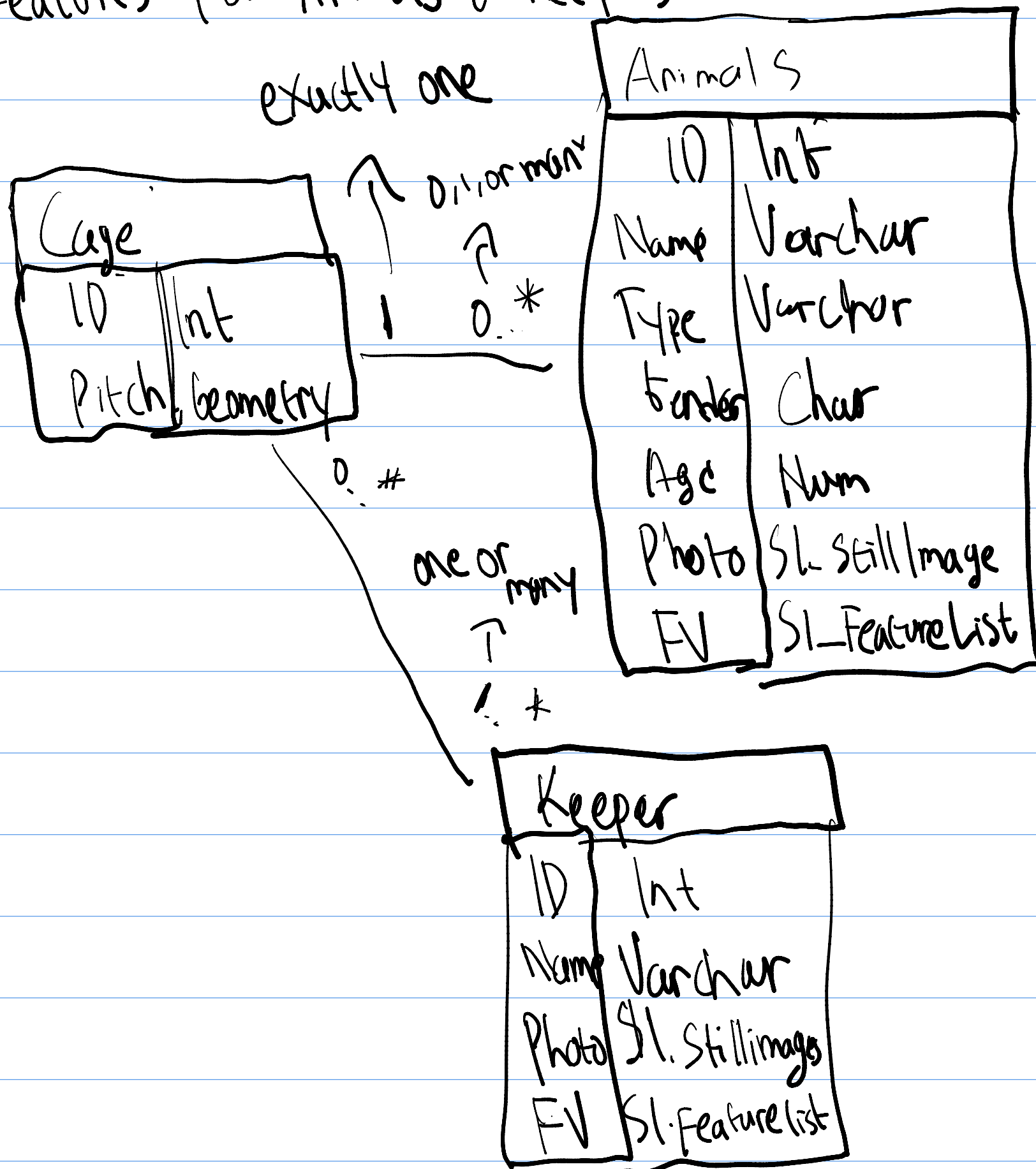
3.1 DB must include

Animals: Name, Gender, Type, Age, Image

Keepers: Name, Pass-photo

Cages: Pitch, and assigned to 1 keeper

SQL/MXL Features For Animals & Keepers



3.2. Create type Animaltype As Object (

ID INT,

Name Varchar(255),

Type Varchar(255),

Gender Char,

Age Num,

Photo SL-StillImage,

FV SL-FeatureList);

Create table Animal of AnimeType(
ID Primary Key
);

Create type KeeperType as object(
ID Int,
Name Varchar(255),
Photo Sl. Still Image,
FV Sl. Feature list);

Create table Keeper of KeeperType(
ID Primary Key
);

Cage:
Animals : 1 to many
Keepers : many to many

Create type AnimNTType as table of ref AnimeType;
Create type KeeperNTType as table of ref KeeperType;

Create table cage(
ID Int,
Pitch ST-Geometry,
Animalsin AnimNTType,
Responsible KeeperNTType);

Nested table Animalsin store as Animalsin_tbl,
Nested table Responsible store as Responsible_tbl.

3.3 Query Content-based Search for Animals

Select photo into myAnim From Animal where ID = "90";

Select t1.ID, t1.Name

from Animal t1, my Animal t2

where t1.FV.SI - score (t2.photo) > 0.5;

4.1 Images + titles, similar to a presented example image, taken in Munich, not bigger than 2048K. Similarity criterion = most important

1. Similarity to an image is the most important

<mpqf:QueryCondition>

<mpqf:Condition xsi:type="mpqf:QueryByMedia"

match type="similar"

pref value="0.8">

<mpqf:MediaUri>URL-to-Example-Image</mpqf:MediaUri>

</mpqf:Condition>

2. Taken in Munich

<mpqf:Condition xsi:type="mpqf:QueryByFreeText"

Preference Value="0.12"

<mpqf:FreeText>City:Munich</mpqf:FreeText>

</mpqf:Condition>

3. File size <= 2048K

<mpqf:Condition xsi:type="mpqf:LessThan">

<mpqf:ArithmeticField typename="Media Format type">/File size

</mpqf:ArithmeticField>

<mpqf:long value>2048</mpqf:long value>

</mpqf:Condition>

</mpqf:QueryCondition>

</mpqf:Input>

4. Outputs Images and their titles

<mpqf:OutputDescription>

<mpqf:ReqField typename="MediaLocator Type">MediaUri</mpqf:ReqField>

<mpqf:ReqField typename="Creation Info Type">/creation/Title</mpqf:ReqField>

</mpqf:OutputDescription>

4.2. Search for images, north: part a mountain, South: Part a lake, dominant

ly (Red 10%, Green 30%, Blue 60%). Must return only JPEG,

Maximum 20 images. Sorted by title similarity ref and file size

1. Input resources

<mpgf:QFDeclaration>

<mpaf:Resource xsi:type="mpgf:MediaResourceType" resourceID="Mountain">

<mpaf:MediaResource>

<mpgf:MediaUri> /UpperArea.png </mpgf:media uri>

</mpgf:MediaResource>

</mpgf:Resource

<mpaf:Resource xsi:type="mpgf:MediaResourceType" resourceID="Lake">

<mpaf:MediaResource>

<mpgf:MediaUri> /LowerArea.png </mpgf:media uri>

</mpgf:MediaResource>

</mpgf:Resource

<mpgf:Resource xsi:type="mpgf:DescResourceType" resourceID="Dom Color">

<mpeg7:Description Unit xsi:type="mpeg7:DominantColorType">

<mpeg7:Colorspace> RGB </mpeg7:Colorspace>

<mpeg7:Value>

<mpeg7:Percentage> 10 </mpeg7:percentage>

<mpeg7:Index> 255 0 0 </mpeg7:Index>

<mpeg7:Color Variance> 0 0 0 </mpeg7:Color Variance>

</mpeg7:Value>

<mpeg7:Value>

<mpeg7:Percentage> 30 </mpeg7:percentage>

<mpeg7:Index> 0 255 0 </mpeg7:Index>

<mpeg7:Color Variance> 0 0 0 </mpeg7:Color Variance>

</mpeg7:Value>

<mpeg7:Value>

<mpeg7:Percentage> 60 </mpeg7:percentage>

<mpeg7:Index> 0 0 255 </mpeg7:Index>

<mpeg7:Color Variance> 0 0 0 </mpeg7:Color Variance>

</mpeg7:Value>

</mpeg7:Description Unit>

</mpaf: Resource>

</mpqf: QF Declaration>

<mpaf: Input>

<mpqf: Query Condition>

North mountain, South lake

<mpqf: Condition xsi:type="mpqf: Spatial Query">

<mpqf: Spatial Relation

RelationType="urn:mpeg:mpqf:cs:SpatialRelation(CS:2000:South"

Source Resource="Lake_1"

target Resource="Mountain_1"

</mpqf: Condition>

Dominant Color Distribution

<mpqf: Condition xsi:type="mpqf: QueryBy Desc">

<mpqf: Desc ResourceREF> Dominant Color 1 </mpqf: Desc ResourceREF>

</mpqf: Condition>

ONLY jpeg

<mpqf: TargetMediaType> image/jpeg </mpqf: TargetMediaType>

</mpqf: Query Condition>

</mpqf: Query Condition>

Returns 20 items and sort

<mpqf: Output Description maxItemCount="20">

<mpqf: SortBy xsi:type="mpqf: Sort BY Field Type">

<mpqf:Field typeName="Content Management Type"/Title </mpqf:Field>

</mpqf: SortBy>

<mpqf: SortBy xsi:type="mpqf: Sort BY Field Type">

<mpqf:Field typeName="Content Management Type"/similarity </mpqf:Field>

</mpqf: SortBy>

<mpqf: SortBy xsi:type="mpqf: Sort BY Field Type">

<mpqf:Field typeName="Content Management Type"/size </mpqf:Field>

</mpqf: SortBy>

</mpgf:OutputDescription>

</mpgf:MpegQuery